1、给定学生成绩，要求使用 for_each 计算：1）每位学生的平均分；2）每门课程的平均分；3）每门课程及格人数；

```
# 输入：
3 // 学生数量
A 66 99 121 120 // 依次为：姓名、物理成绩、语文成绩、英语成绩、数学成绩
B 55 105 111 121
C 60 120 89 140

# 输出：
学生平均分：
A：101.5
B：
C：

课程平均分：
物理：60.3333333
语文：
英语：
数学：

课程及格情况：// 注意物理满分为100（60及格），其他满分150（90及格）
物理：2/3
语文：
英语：
数学：
```

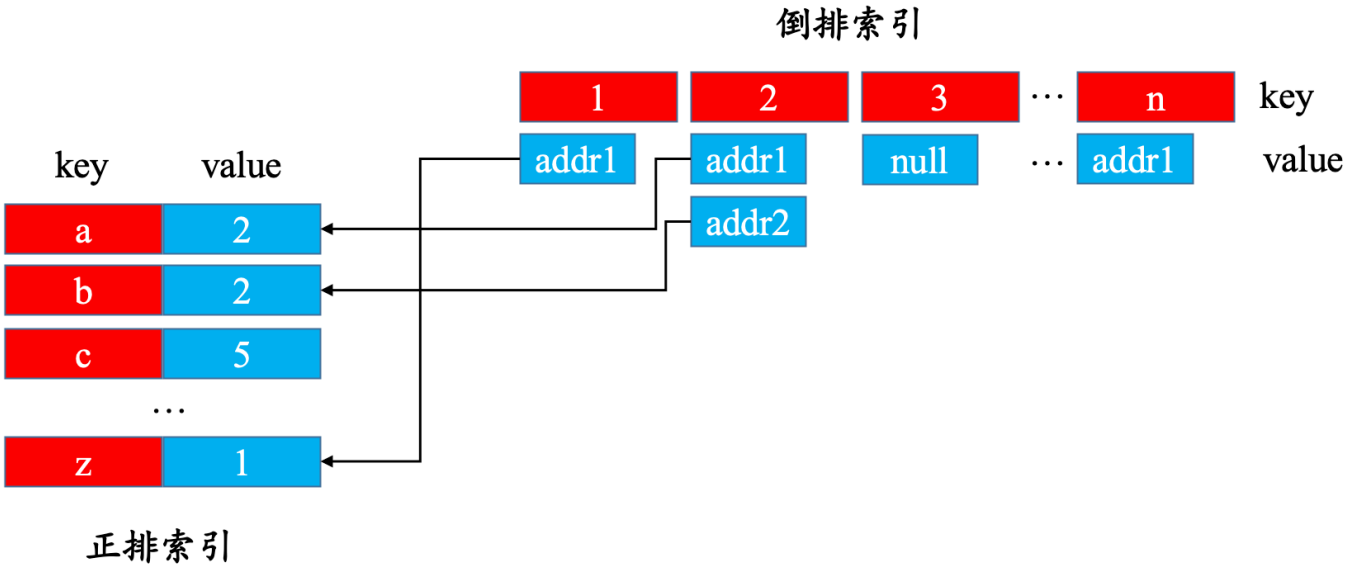
2、合并链表 Plus Pro：合并多个升序链表，要求每个链表仅遍历一次

```
# 输入：
3 // 链表数量
1->7->9->10
2->3->4
0->5->6

# 输出：
0->1->2->3->4->5->6->7->9->10

# 注意：要求每个链表仅能遍历一次，即遍历次数 = 所有链表节点数量之和
```

3、补全类的定义，要求为：输入一个文档，统计文档中出现的单词词频，保存为 map<string, int> 类型的变量（称为正排索引）；基于正排索引，实现倒排索引，类型为 map<int, vector<map<string, int>::iterator>>。当输入一组查询请求（以文件的形式），要求输出查询结果



- 数据信息

```
# 文档内容：
SOTA methods for learning cross-lingual word embeddings have relied on bilingual dictionaries or parallel corpora. Recent studies showed that the need for parallel data supervision can be alleviated with character-level information. While these methods showed encouraging results, they are not on par with their supervised counterparts and are limited to pairs of languages sharing a common alphabet. In this work, we show that we can build a bilingual dictionary between two languages without using any parallel corpora, by aligning monolingual word embedding spaces in an unsupervised way. Without using any character information, our model even outperforms existing supervised methods on cross-lingual tasks for some language pairs. Our experiments demonstrate that our method works very well also for distant language pairs, like English-Russian or English- Chinese. We finally describe experiments on the English-Esperanto low-resource language pair, on which there only exists a limited amount of parallel data, to show the potential impact of our method in fully unsupervised machine translation. Our code, embeddings and dictionaries are publicly available.

# 查询文件内容：
data
language
information
5
3

# 查询结果内容类似于：
2 // data 对应的出现次数
3
2
```

```
on ... // 出现次数为5的单词列表
language to ...
```

- 类定义

```
class Index {
private:
  // 正排索引
  map<string, int> index;
  // 倒排索引
  map<int, vector<map<string, int>::iterator>> rindex;

public:
  // 在初始化过程中，构造倒排索引
  Index(const string& str);
  Index(ifstream& in);
  Index();

  // 查询单词str的词频（利用正排索引）
  int operator[](const string& str);

  // 查询词频为freq的单列列表（利用倒排索引）
  vector<string> operator[](const int& freq);

  // 从文件中读取索引信息
  friend ifstream& operator>>(ifstream &in, Index& index);

  // 将索引信息保存到文件中
  friend ofstream& operator<<(ofstream &out, Index& index);
}
```